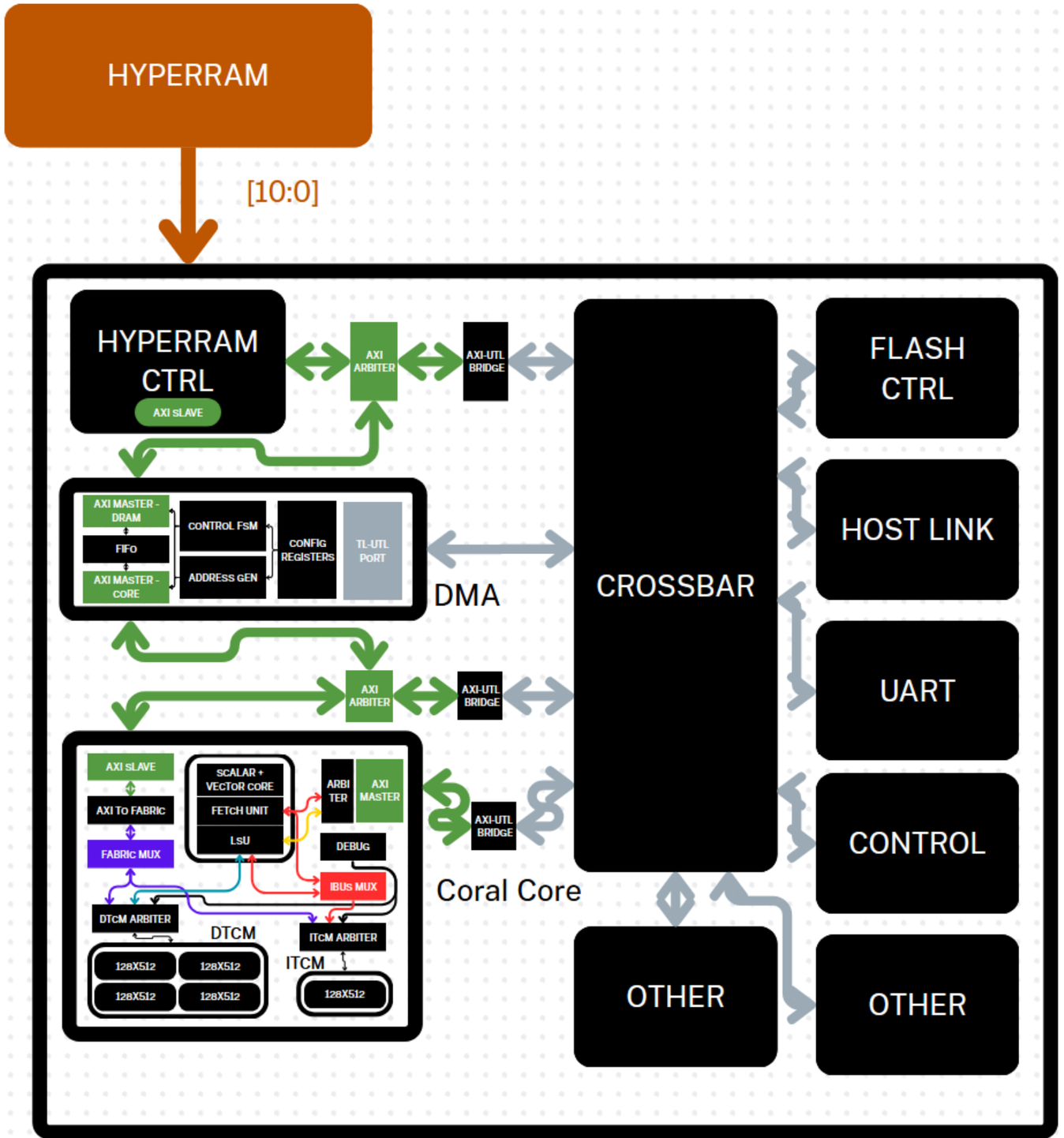
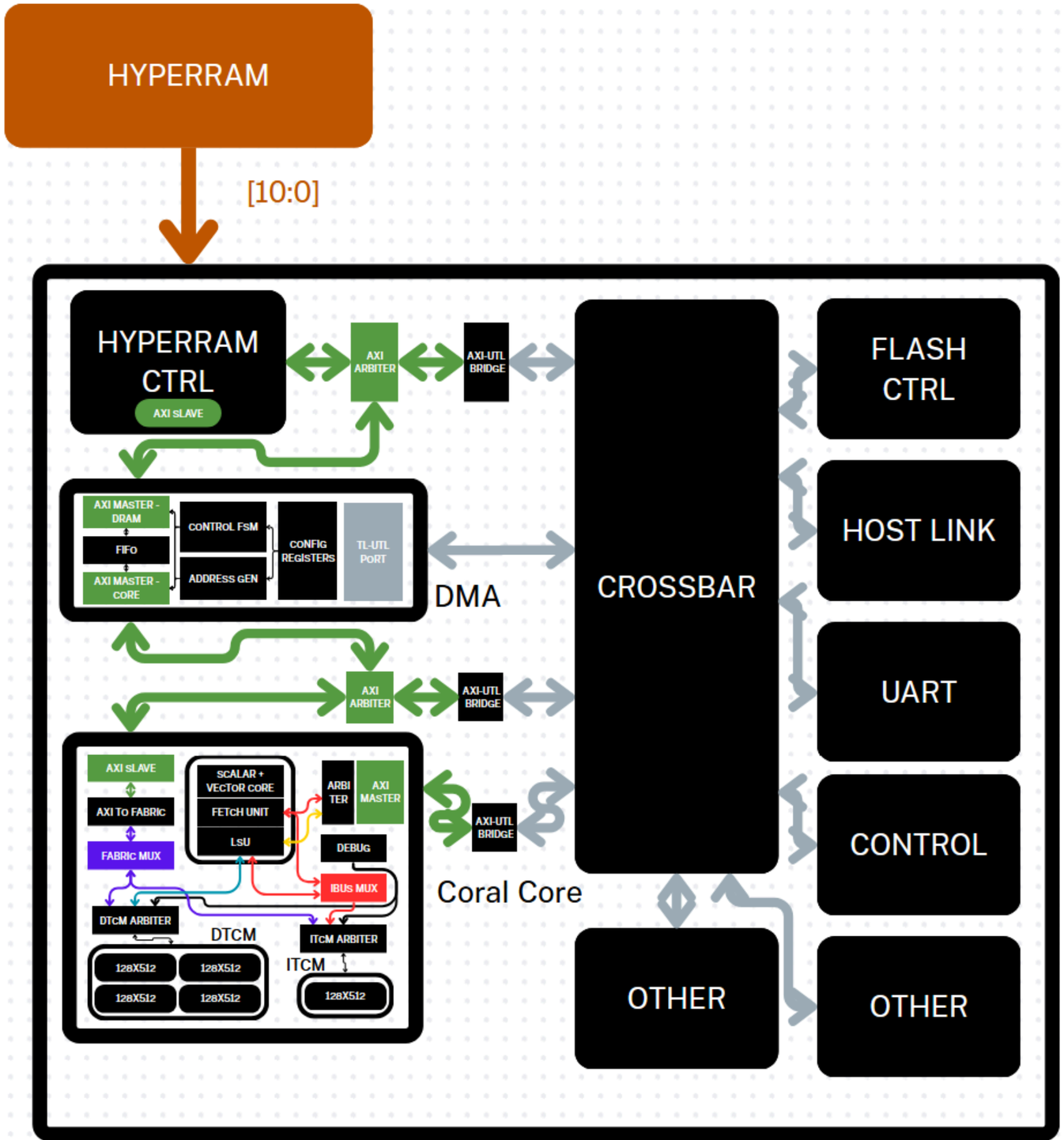
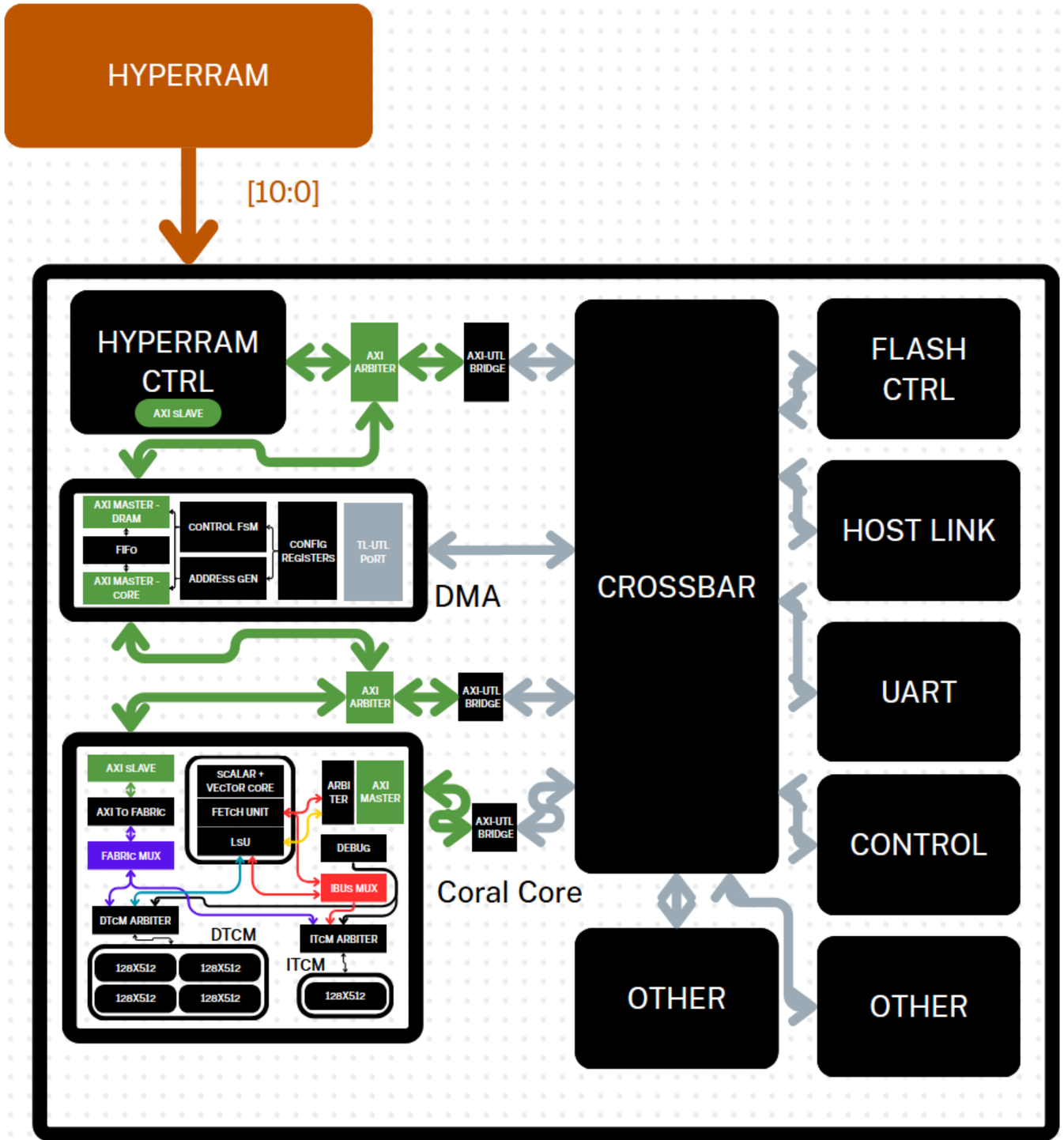
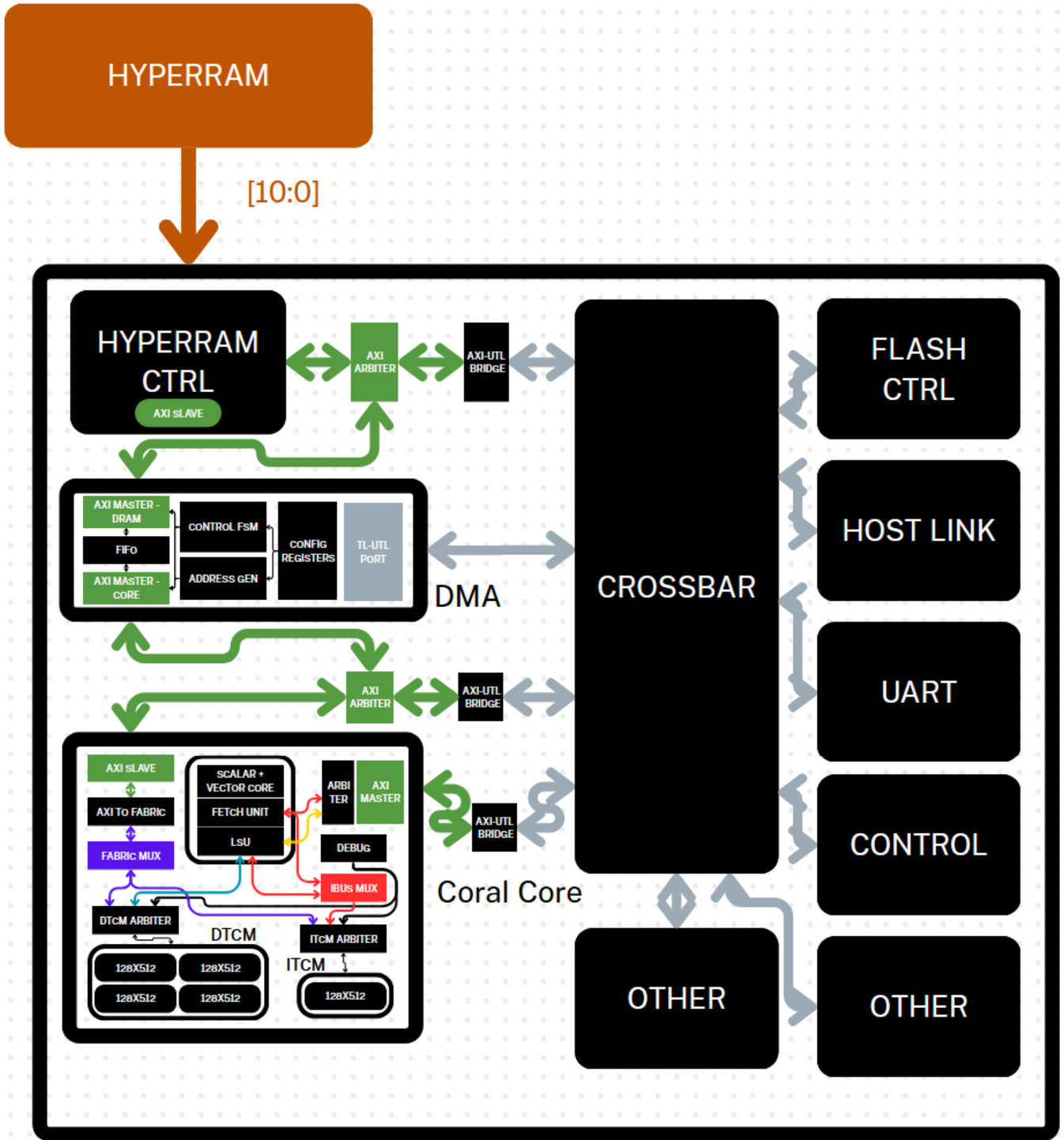

HYPERRAM
[10:0]
HYPERRAM CTRL
AXI sLAVE
AXI ARBITER
AXI-UTL BRIDGE
AXI MASTER - DRAM
FIFO
AXI MASTER - CORE
CONTROL FSM
ADDRESS GEN
CONFIG REGISTERS
TL-UTL PORT
DMA
AXI ARBITER
AXI-UTL BRIDGE
AXI sLAVE
AXI To FABRIC
FABRIC MUX
SCALAR + VECTOR CORE
FETCH UNIT
LSU
ARBI TER
AXI MASTER
DEBUG
IBUs MUX
AXI-UTL BRIDGE
Coral Core
DTCM ARBITER
DTCM
ITCM ARBITER
ITCM
128X512
128X512
128X512
128X512
128X512
CROSSBAR
FLASH CTRL
HOST LINK
UART
CONTROL
OTHER
OTHER


HYPERRAM
[10:0]
HYPERRAM CTRL
AXI sLAVE
AXI ARBITER
AXI-UTL BRIDGE
AXI MASTER - DRAM
FIFO
AXI MASTER - CORE
CONTROL FSM
ADDRESS GEN
CONFIG REGISTERS
TL-UTL PORT
DMA
AXI ARBITER
AXI-UTL BRIDGE
AXI sLAVE
AXI To FABRIC
FABRIC MUX
SCALAR + VECTOR CORE
FETCH UNIT
LSU
ARBI TER
AXI MASTER
DEBUG
IBUs MUX
AXI-UTL BRIDGE
Coral Core
DTCM ARBITER
DTCM
128X512
128X512
128X512
128X512
ITCM ARBITER
ITCM
128X512
CROSSBAR
FLASH CTRL
HOST LINK
UART
CONTROL
OTHER
OTHER

Power on

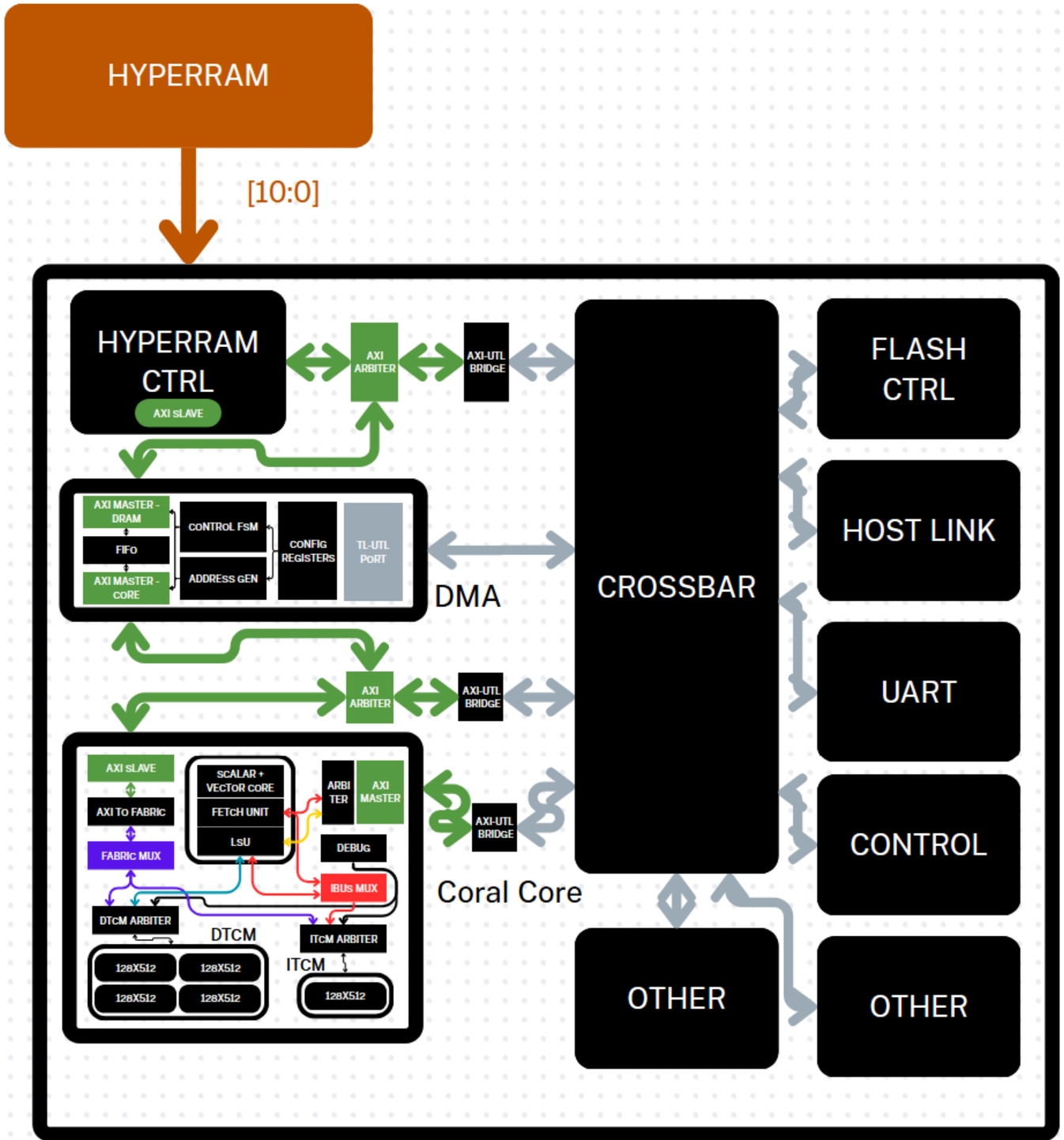

HYPERRAM
[10:0]
HYPERRAM CTRL
AXI sLAVE
AXI ARBITER
AXI-UTL BRIDGE
AXI MASTER - DRAM
FIFO
AXI MASTER - CORE
CONTROL FSM
ADDRESS GEN
CONFIG REGISTERS
TL-UTL PORT
DMA
AXI ARBITER
AXI-UTL BRIDGE
AXI sLAVE
AXI To FABRIC
FABRIC MUX
SCALAR + VECTOR CORE
FETCH UNIT
LSU
ARBI TER
AXI MASTER
DEBUG
IBUs MUX
AXI-UTL BRIDGE
Coral Core
DTCM ARBITER
DTCM
128X512
128X512
128X512
128X512
ITCM ARBITER
ITCM
128X512
CROSSBAR
FLASH CTRL
HOST LINK
UART
CONTROL
OTHER
OTHER

Reset Held



Initial values

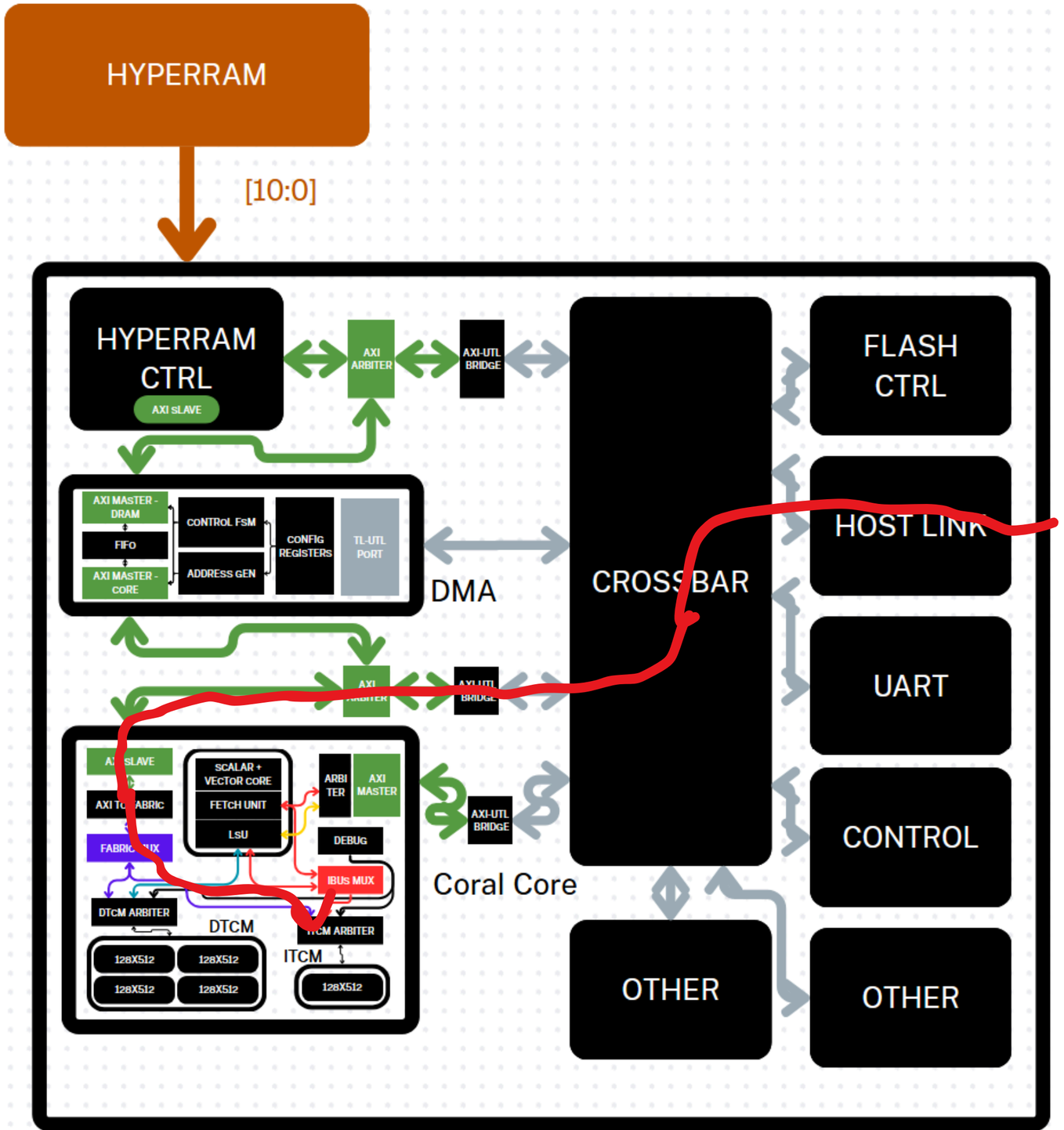
Block	State after reset
Coral core	**Frozen:** `RESET_CONTROL = 0x3` (held in reset, clock gated). `PC_START ← boot_addr`, which you tie to the ITCM base, `0x0`. `STATUS = 0`. `axi_slave` ignores the first cycle.
ITCM / DTCM	Powered but **full of random bits.** SRAM powers up in random states and nothing clears it.
Crossbar, bridges, muxes	Idle, no transactions in flight
Burst DMA	All registers 0, FSM in **IDLE**
HyperRAM controller	In reset, not talking to the chip yet
Flash controller	In reset
CLINT	Timer starts counting from 0
UART, GPIO	UART idle; GPIO pins are inputs



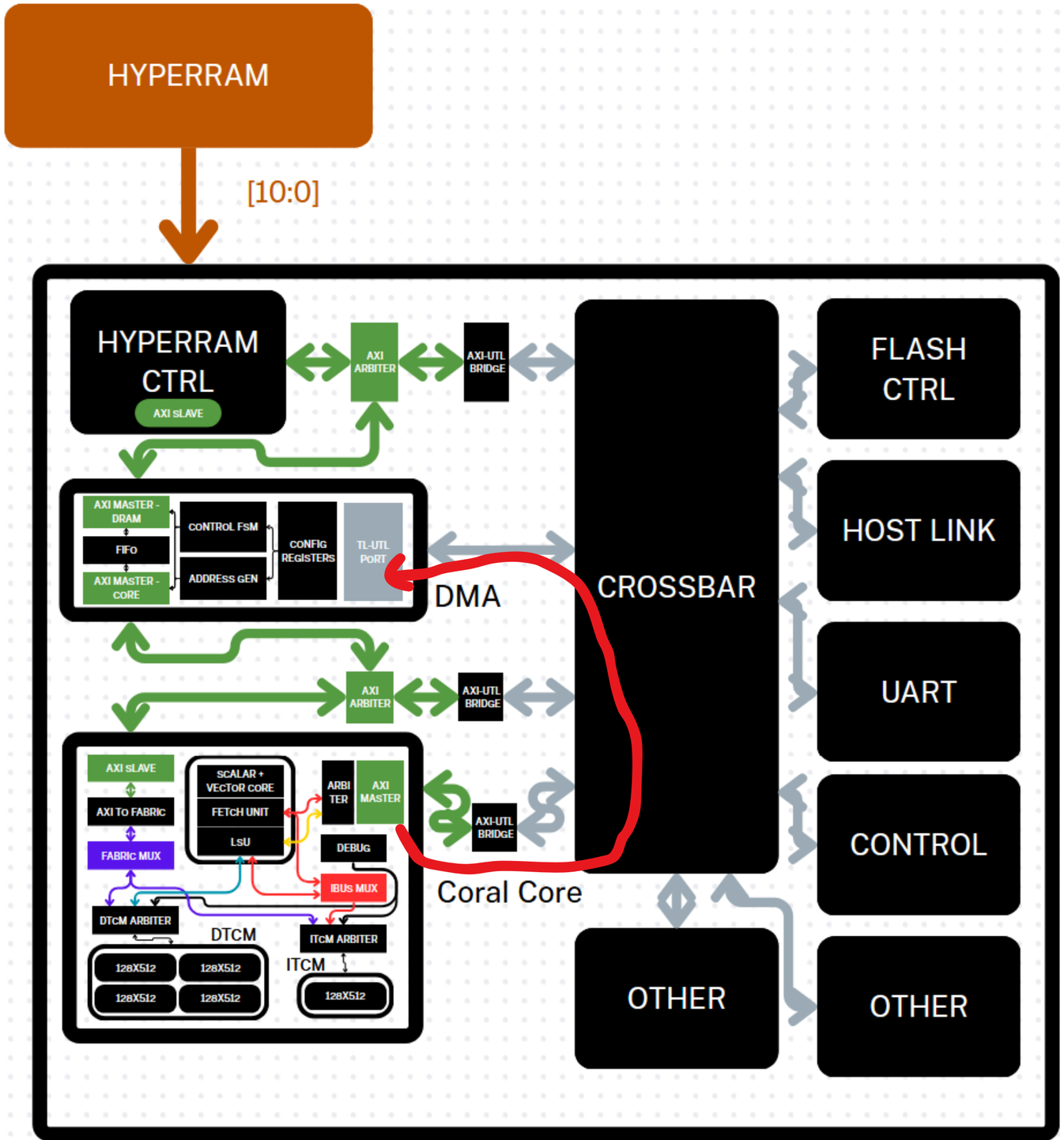
HYPERRAM power on sequence

FLASH power on sequence

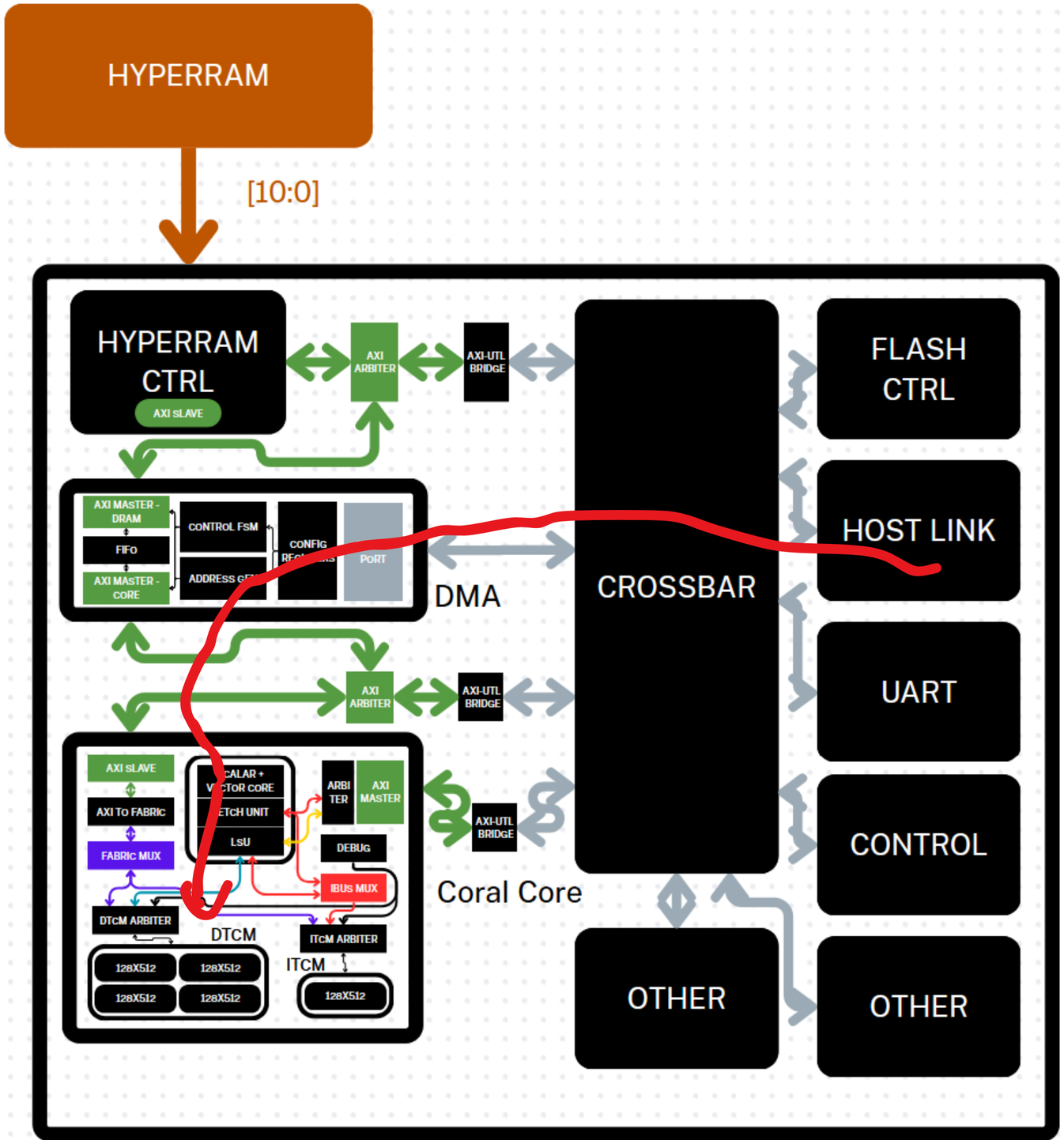
Flash Option A : Host BOOT


HYPERRAM
[10:0]
HYPERRAM CTRL
AXI SLAVE
AXI ARBITER
AXI-UTL BRIDGE
AXI MASTER - DRAM
FIFO
AXI MASTER - CORE
CONTROL FSM
ADDRESS GEN
CONFIG REGISTERS
TL-UTL PORT
DMA
CROSSBAR
FLASH CTRL
HOST LINK
UART
CONTROL
OTHER
OTHER
SCALAR + VECTOR CORE
FETCH UNIT
LSU
ARBI TER
AXI MASTER
DEBUG
IBUS MUX
DTCM ARBITER
DTCM
128X512
128X512
128X512
128X512
ITCM
128X512
AXI-UTL BRIDGE
Coral Core

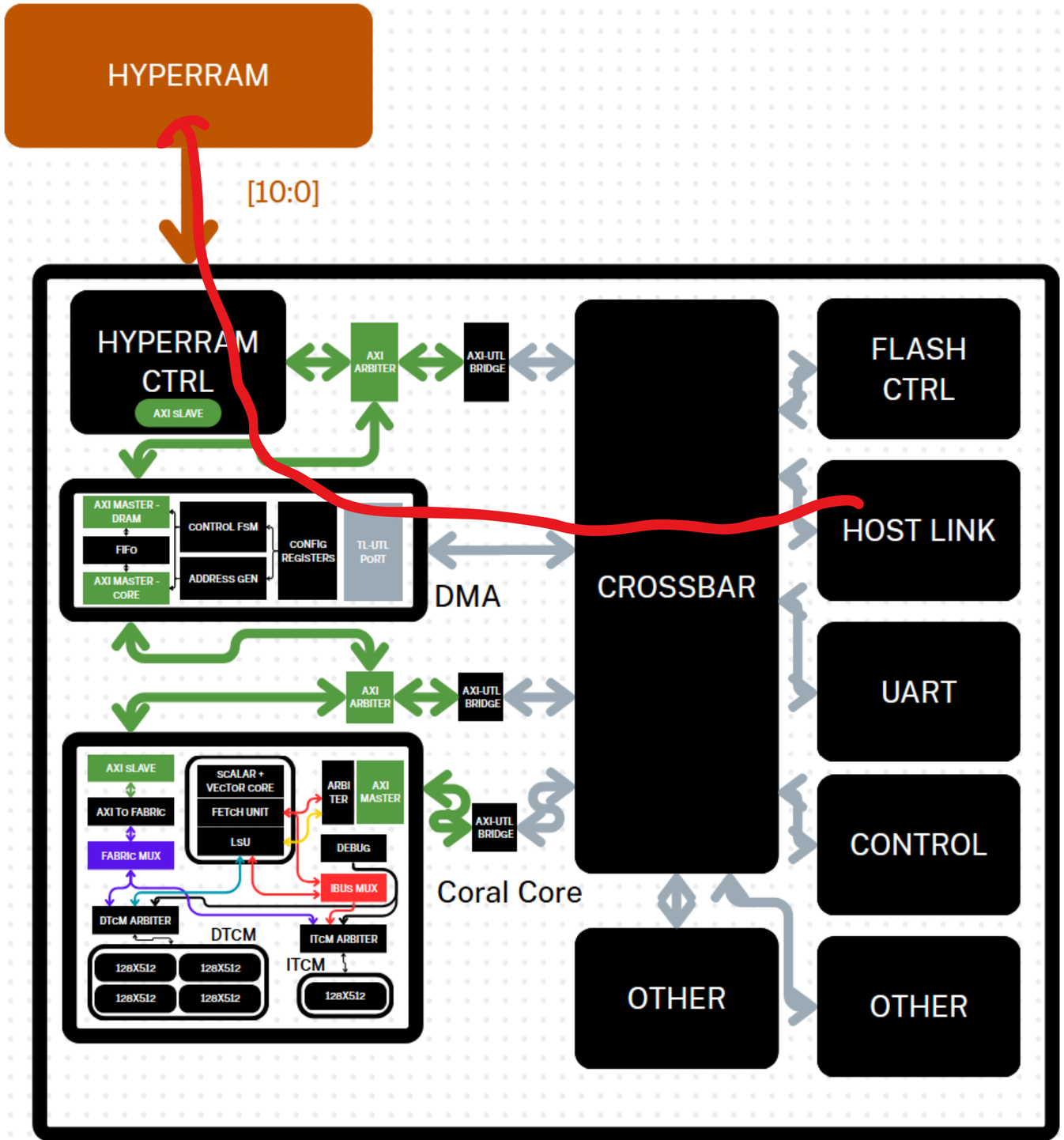
ITCM Load



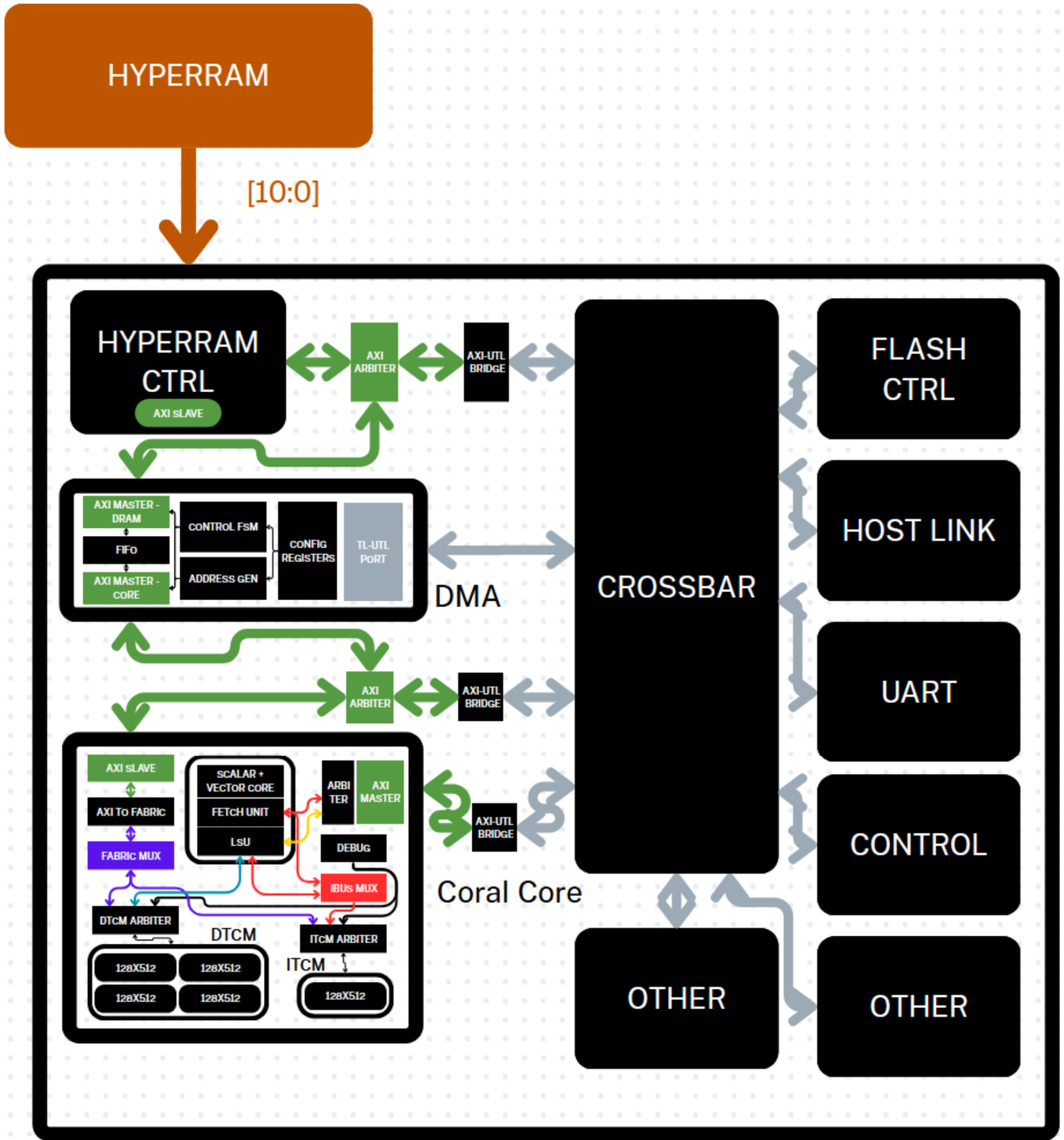
Now the ITCM starts executing and makes requests to the DMA



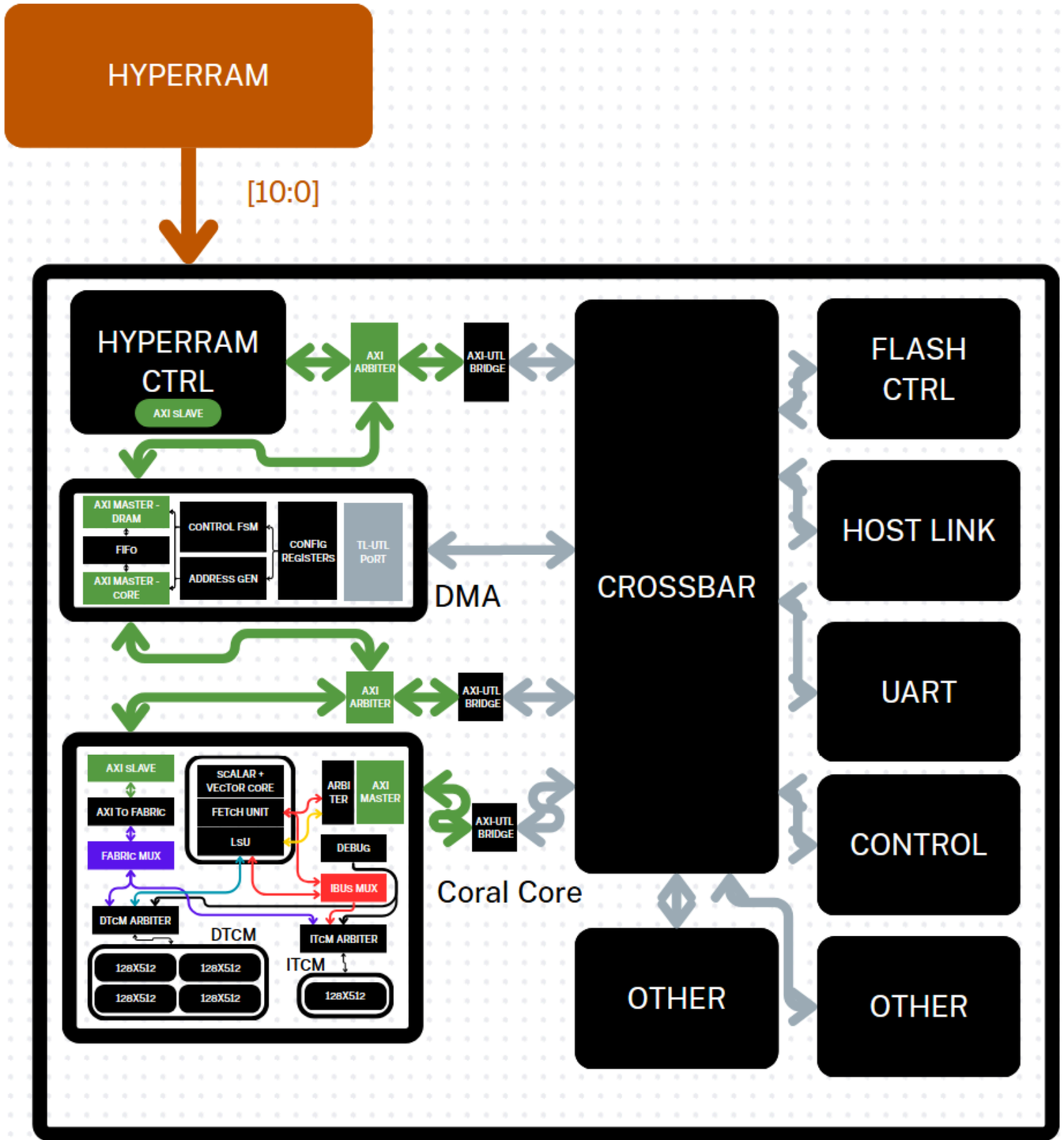
DMA says get data from ctrller to DTCM



DMA says get data from ctrller to HYPERRAM



At the same time, the hardware modules are running Boot FSMs.



Then you start putting your kernels in ITCM and start executing.

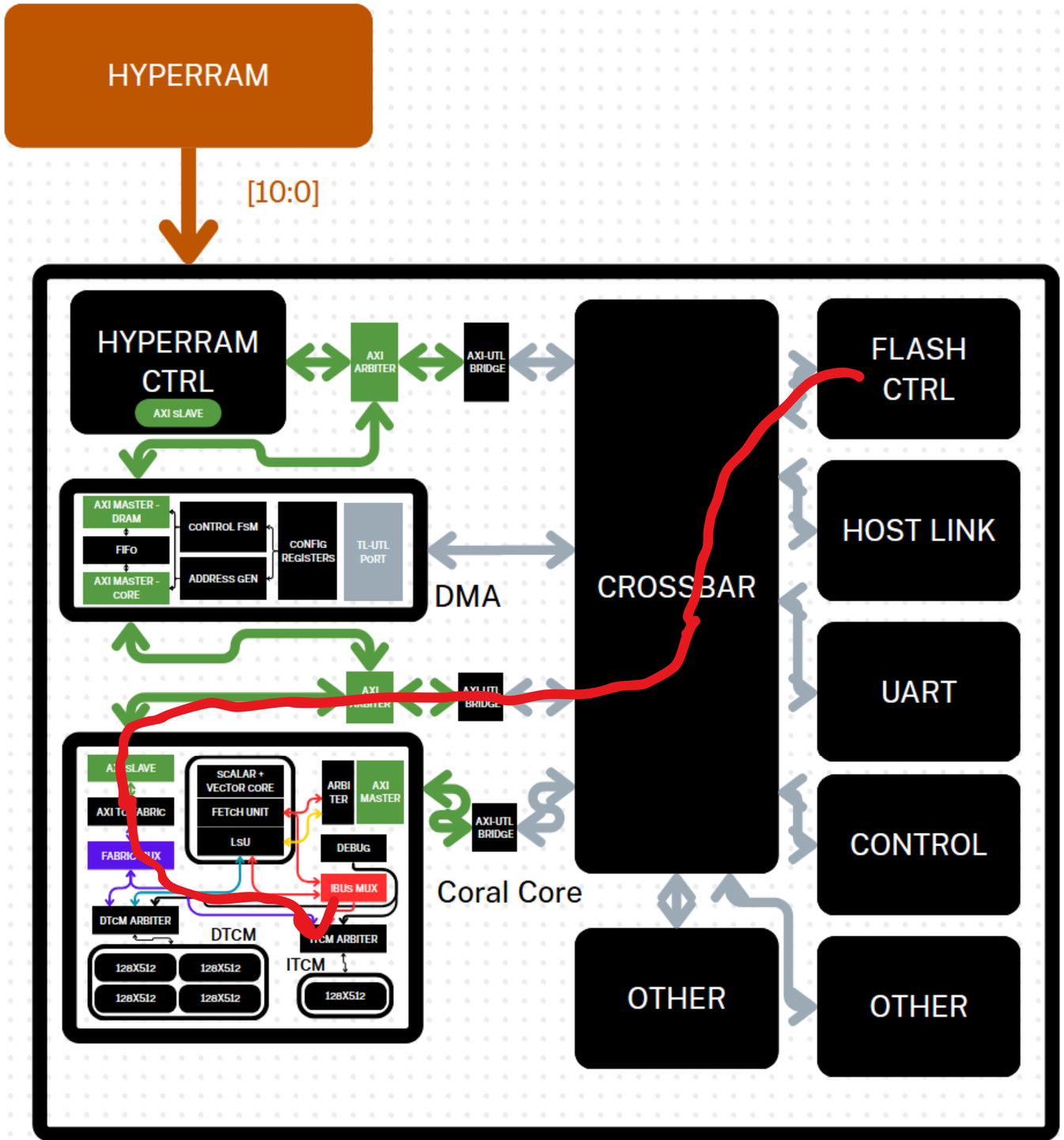
Flash Option A : FLASH BOOT

(n lines)

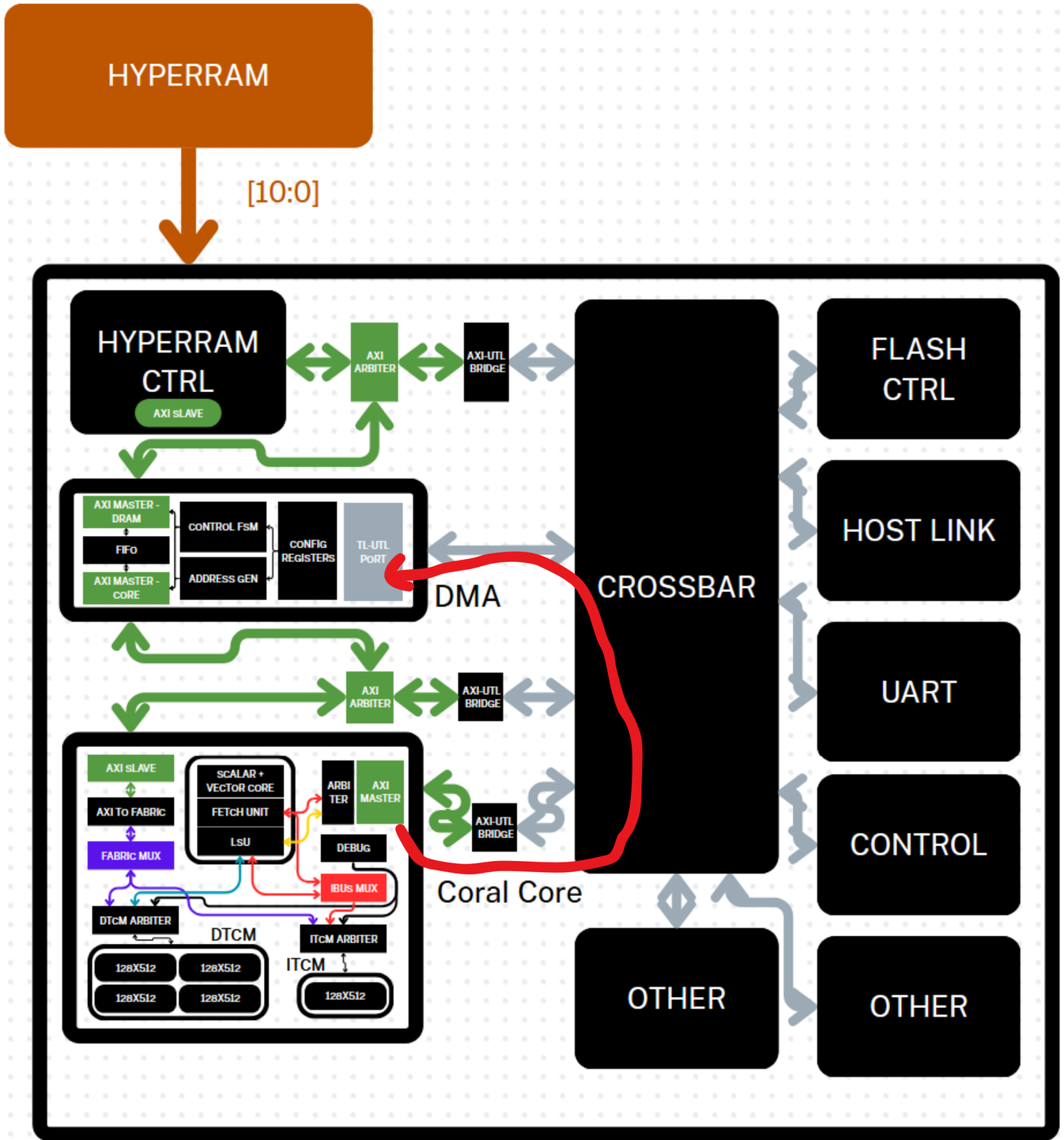
Host-Lin(SPI): 4·n mb/s

QSPI flash: 16 mb/s

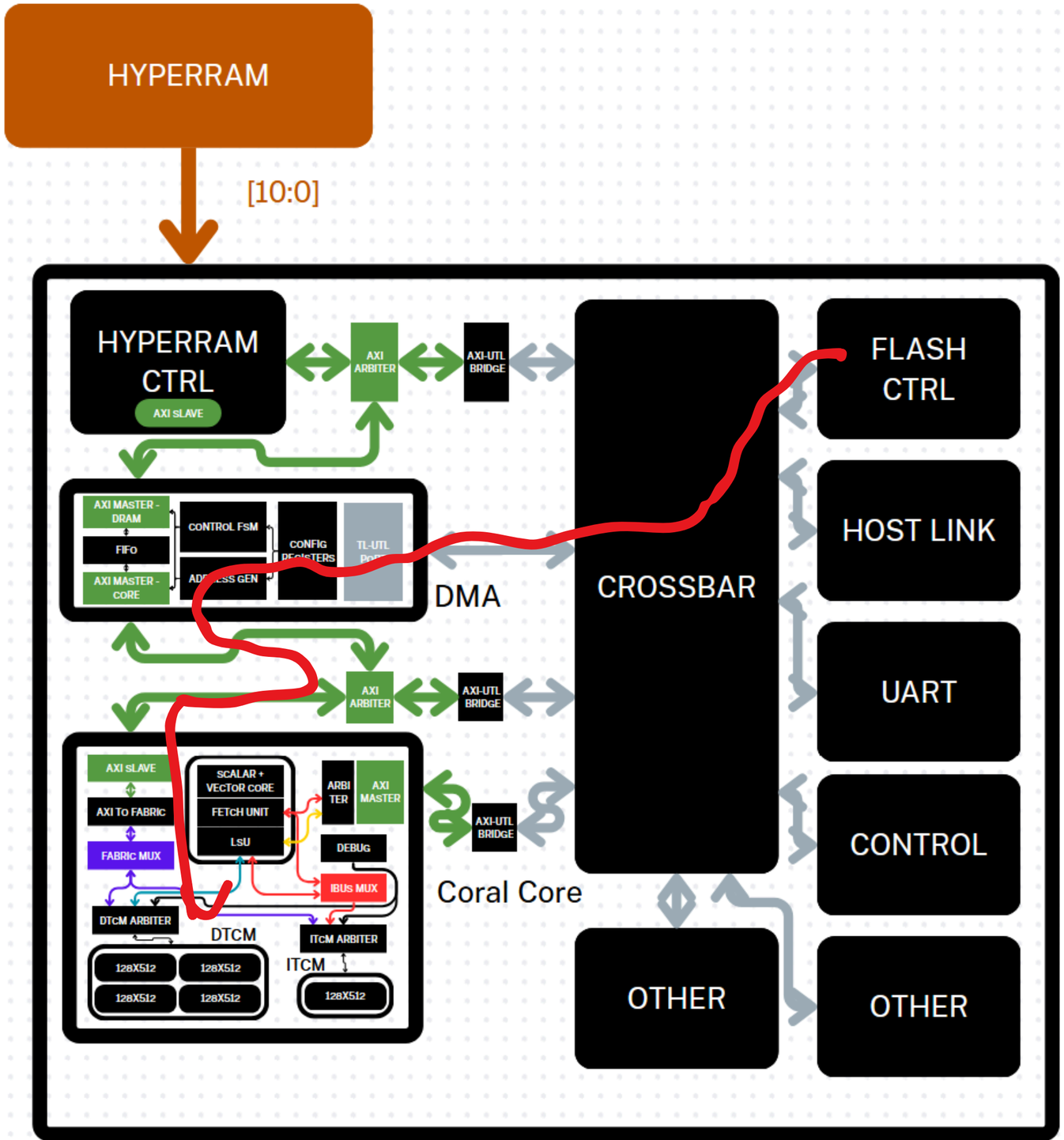
HYPERRAM : 97 mb/s


HYPERRAM
[10:0]
HYPERRAM CTRL
AXI SLAVE
AXI ARBITER
AXI-UTL BRIDGE
AXI MASTER - DRAM
FIFO
AXI MASTER - CORE
CONTROL FSM
ADDRESS GEN
CONFIG REGISTERS
TL-UTL PORT
DMA
CROSSBAR
FLASH CTRL
HOST LINK
UART
CONTROL
OTHER
OTHER
SCALAR + VECTOR CORE
FETCH UNIT
LSU
ARBI TER
AXI MASTER
DEBUG
IBUs MUX
DTCM ARBITER
DTCM
ITCM
128X512
128X512
128X512
128X512
128X512
AXI-UTL BRIDGE
Coral Core

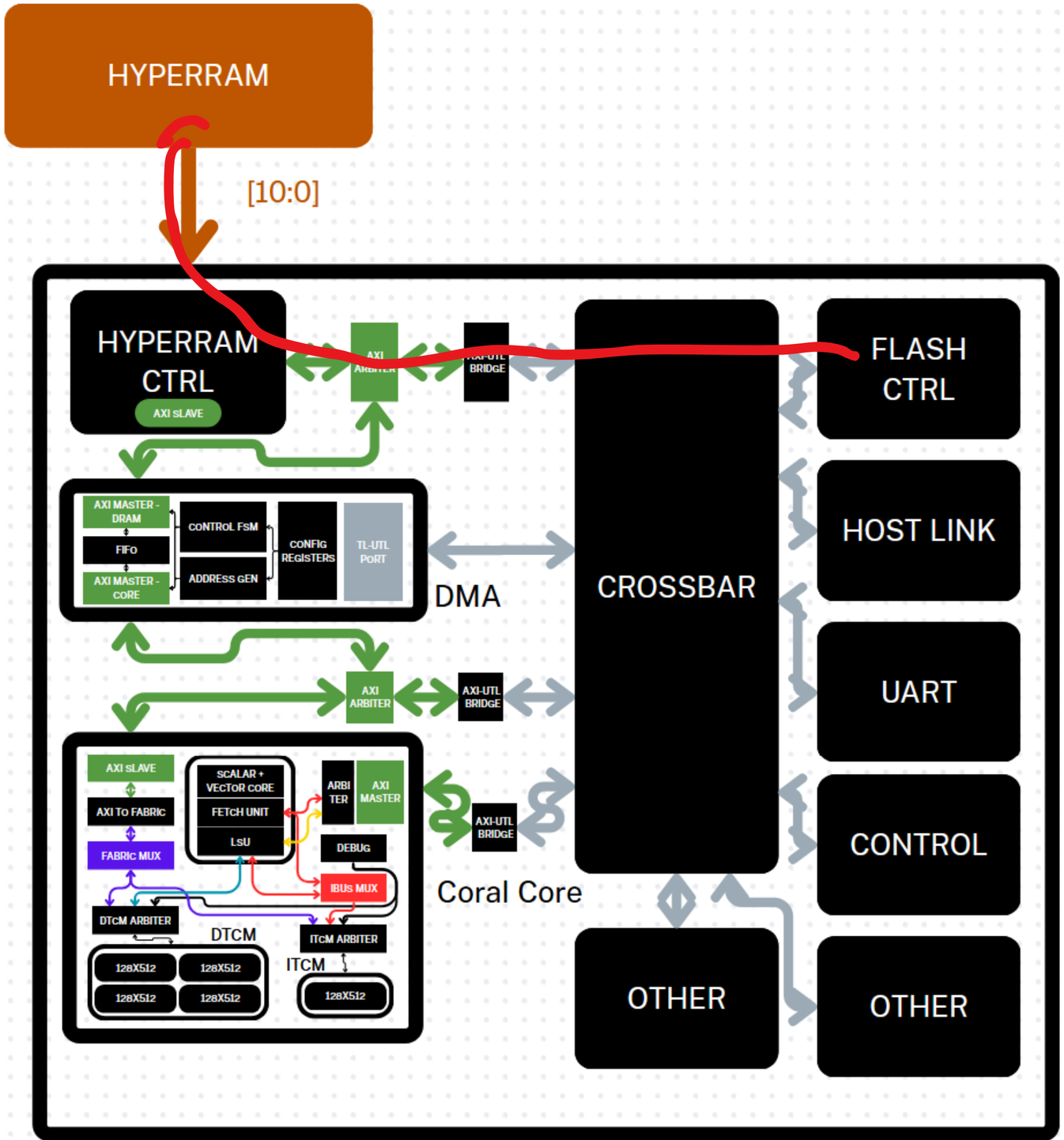
ITCM Load



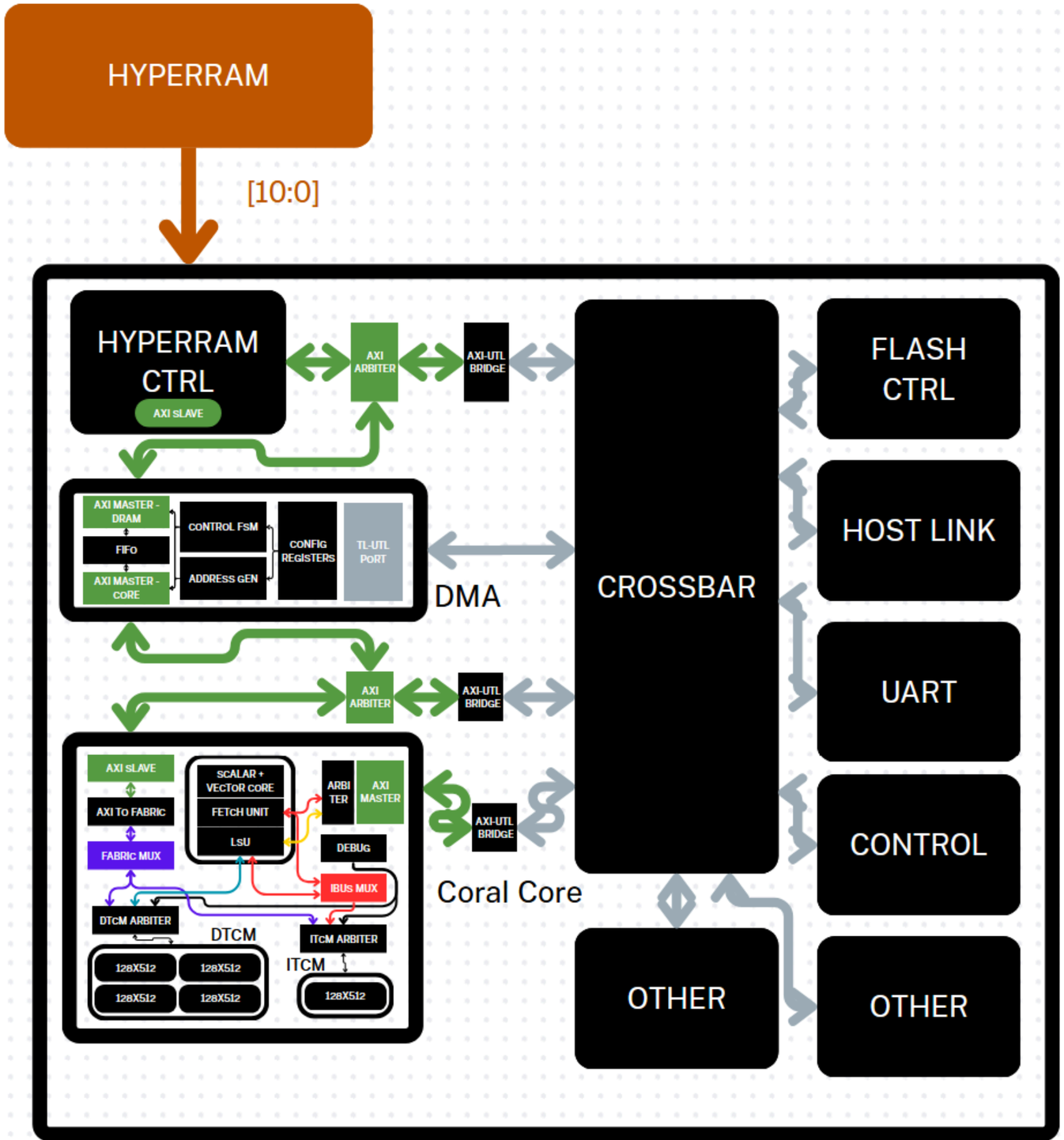
Now the ITCM starts executing and makes requests to the DMA



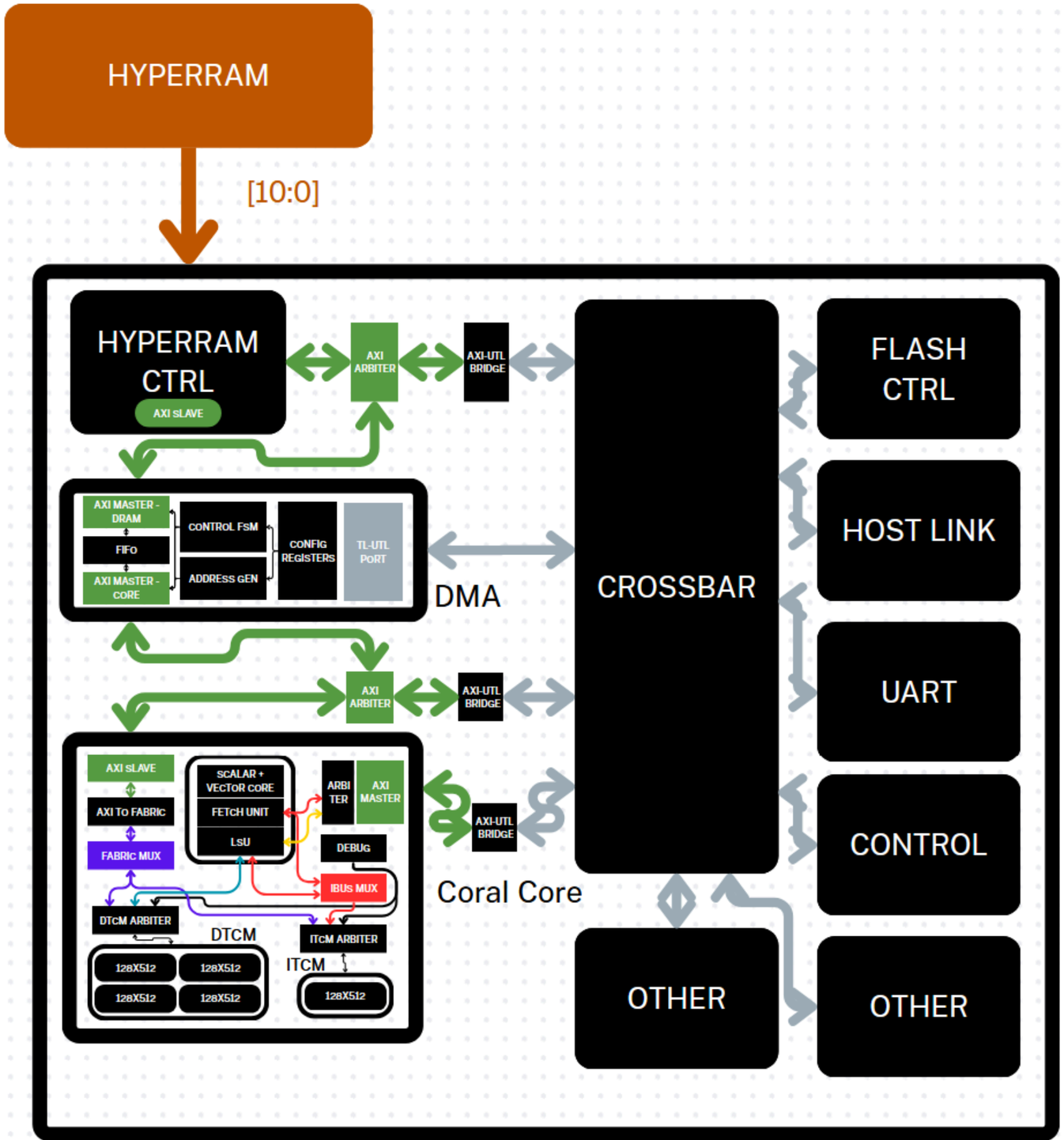
DMA says get data from ctrller to DTCM



DMA says get data from ctrller to HYPERRAM



At the same time, the hardware modules are running Boot FSMs.



Then you start putting your kernels in ITCM and start executing.

Execution

#	Kernel	What it does	Where it's used	Share of time
1	**GEMV**	Matrix × vector: multiply-accumulate int8 weights against the activation vector	QKV, output projection, MLP up/down, attention scores, output layer	**~90%**
2	**Requantize**	Shrinks int32 results back to int8 with a per-row scale	After every GEMV	~2%
3	**LayerNorm**	Normalizes a vector to mean 0 and variance 1, then applies a learned scale and shift	Before attention, before the MLP, and once at the end	~2%
4	**Softmax**	Turns attention scores into weights that sum to 1	Inside attention, per head	~1%
5	**Weighted sum (axpy)**	Adds up V vectors, each multiplied by its attention weight	Inside attention, per head	~1–2%
6	**GELU**	Activation function, done as a 256-entry lookup table	Between MLP up and MLP down	<1%
7	**Residual add**	Adds a block's output back onto the running vector `x`	After attention and after the MLP	<1%
8	**Argmax / top-k**	Finds the highest-scoring word(s)	Picking the next token	<1%
9	**DMA streaming**	Double-buffered tile loading and the done-flag sync	Wrapped around every GEMV, plus KV cache writes and the embedding lookup	Overlapped with compute

Kernel 1: GEMV, one output row

What it computes: `y = W[0]·x[0] + W[1]·x[1] + … + W[383]·x[383]`

Pseudo-code:

```
acc[0..15] = 0                          # 16 running sums, int32
repeat 24 times:                        # 24 × 16 = 384 elements
    w[0..15] = load 16 weights from DTCM
    x[0..15] = load 16 activations from DTCM
    p[i]     = w[i] * x[i]              # 16 products, as int16
    acc[i]  += p[i]                     # add into the int32 sums
    move both pointers forward 16 bytes
y = acc[0] + acc[1] + … + acc[15]       # combine the 16 lanes
```

Instruction by instruction:

Instruction	Plain meaning
`vsetvli t0, zero, e32, m4`	Mode: 32-bit elements, 4 registers grouped (16 lanes)
`vmv.v.i v8, 0`	`acc[0..15] = 0` (in v8–v11)
loop:	
`vsetvli t0, a2, e8, m1`	Mode: 8-bit, 1 register (16 lanes). `t0` = how many this round (16).
`vle8.v v0, (a0)`	`w[0..15] = memory[a0 … a0+15]` (weights)
`vle8.v v1, (a1)`	`x[0..15] = memory[a1 … a1+15]` (activations)
`vwmul.vv v2, v0, v1`	`p[i] = w[i] * x[i]`, widened to int16 (fills v2–v3)
`vsetvli zero, t0, e16, m2`	Mode: 16-bit, 2 registers grouped
`vwadd.wv v8, v8, v2`	`acc[i] = acc[i] + p[i]`, widening p to int32
`add a0, a0, t0`	weight pointer += 16
`add a1, a1, t0`	activation pointer += 16
`sub a2, a2, t0`	elements left −= 16
`bnez a2, loop`	if elements left ≠ 0, go back to **loop**
`vsetvli t0, zero, e32, m4`	Mode: 32-bit, 4 grouped
`vmv.s.x v12, zero`	`start = 0`
`vredsum.vs v12, v8, v12`	`total = start + acc[0] + … + acc[15]`
`vmv.x.s a0, v12`	Move `total` into scalar register `a0` (the answer)